Al-Akhir Nayan

PSID: 2371302

Multi-issue out-of-order execution

Programming assignment report

Overview

I extended the starting Tomasulo simulator so it can issue one to four instructions per cycle, in program order. Instructions still go through a reorder buffer, register renaming and branch prediction, so they execute out of order but commit in order. I tested it with fourteen programs at every issue width, and all of them end in the same state as a simple in-order interpreter. The sections below cover what I changed, how the issue width affects execution time, and the modelling decisions I made.

Loads follow the load/store queue rule from the assignment: if any older store address is unknown, a load reads memory even when another store has a known matching address. Once all older addresses are known, it can forward from the youngest matching store. Stores check for stale loads both when their address resolves and when they commit. If a conflict is found, the load and everything after it are squashed and run again.

AI use: I used an AI assistant (Claude) to help with the code and this report, as permitted for this assignment.

Assignment 2: Tomasulo algorithm with multiple issue

How to run

Requires Python 3 (no packages).

```
cd code
python3 main.py                          # runs test_case.txt
python3 main.py tests/03_loop_branch.txt     # any input file
python3 main.py tests/03_loop_branch.txt -w 4 -s
python3 run_tests.py                     # correctness check, all tests, widths 1-4
python3 -m unittest -v test_memory_rules     # memory-rule regression checks
```

Options:

option	meaning
-w N	issue width 1–4 (overrides Issue width = N in the file; default 1)
-c N	number of CDBs (overrides CDB width = N; default 1)
-s	also list squashed instructions (wrong path or memory-order replay)

Input file format

Here is a complete input file that runs as-is. The optional settings are marked with # comments.

```
                # of rs        Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4             1               1
FP adder        3             4               1
FP multiplier   2             15              1
Load/store unit 5             1               5               1

ROB entries = 64
Issue width = 2         # optional, 1-4, default 1
CDB width = 1           # optional, default 1
Commit width = 2        # optional, default = issue width
R1=8, F20=3.0
Mem[4]=3.0, Mem[8]=2.0

Ld F2 0(R1)
Mult.d F4 F2 F20
Addi R1 R1 -4
Bne R1 R0 -4            # target = PC + 4 + offset*4
```

Text after # is a comment. Registers and memory start at zero.

1. What was added

The original code handled one instruction per cycle, and that assumption was built into several of the pipeline stages. I kept the same files (one per stage) but moved all processor state into a single Machine object, so each stage can handle several instructions in a cycle when resources allow. I also added input-file configuration, branch prediction and the final register and memory output. The table summarises the changes.

feature	where
Issue width 1–4 (Issue width = N in the input or -w N). Up to N instructions are issued per cycle, strictly in program order. Issue stops at the first instruction that has no free ROB entry, reservation station or load/store queue entry. Instructions issued in the same cycle can depend on each other, because the RAT is updated after each one.	issue.py

feature	where
Commit width (default = issue width) and configurable number of CDBs (default 1, as in the spec). The oldest finished instructions get the CDBs first.	commit.py, wb.py
Configuration from the input file: RS counts, EX/MEM cycles, number of FUs per type, ROB size, initial registers and memory. Before this, the simulator only read a list of instructions.	init.py
Branch prediction: a 1-bit predictor stored in an 8-entry BTB, indexed by the low 3 bits of the word address. Prediction happens at issue, and a predicted-taken branch ends the issue group. The branch is resolved at the end of EX. On a misprediction, all younger instructions are squashed from the ROB, reservation stations, load/store queue, busy units and memory port. The RAT is rebuilt from the surviving ROB entries, and the correct fetch starts at cycle n+2.	issue.py, exe.py
Multiple functional units per type (# of FUs column). The integer adder and the address adders are unpipelined; the FP adder and FP multiplier are pipelined (one new operation per unit per cycle).	init.py, exe.py
Load/store queue as specified. A load accesses memory if any older store address is unresolved, even when another store is a known match. If all older addresses are known, it forwards from the youngest matching store in one cycle once the data is ready, or reads memory if there is no match. Stores write memory at commit and check for stale younger loads both when their address resolves and when they commit. Conflicting loads and all younger instructions are squashed and refetched.	mem.py, exe.py, commit.py
Output: instruction status table, all registers, non-zero memory, plus cycles, IPC and branch statistics.	print_status.py
Correctness checks: compare final registers, memory and committed instruction order with a sequential interpreter at widths 1–4; separately test unresolved-store priority, replay and forwarding.	run_tests.py, test_memory_rules.py

1.1 Issue several instructions in program order

Code Changes. The excerpts below compare original_code/ with the submitted code/. They show the main changes; the complete updated files are in Appendix A and in the repository. Because processor state was moved into a Machine object, these are comparisons of the same responsibilities rather than one-line patches.

Initial Version

original_code/issue.py (excerpt)

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    op = ins.split(' ')[0]
```

My Version

code/issue.py (excerpt)

```
def issue(m, cycle):
    if cycle < m.fetch_resume:
        return
    for _ in range(m.cfg.issue_width):
        if m.pc >= len(m.program):
            return
        instr = m.program[m.pc]
        # structural hazards stall issue (in order)
        if len(m.rob) >= m.cfg.rob_size:
            return
        if instr.unit == 'ldsd':
            if len(m.ld_sd_queue) >= m.cfg.rs['ldsd']:
                return
        else:
            index = check_rs_space(m.rs[instr.unit])
            if index < 0:
                return

        entry = ROB_entry(m.next_seq, 'ROB%d' % (m.next_seq % m.cfg.rob_size), instr)
        m.next_seq += 1
```

Explanation

The original issue function fetches one instruction each time it is called. My version loops up to the configured issue width and stops as soon as the next instruction cannot obtain a ROB entry or its required queue or reservation station. It does not skip a blocked instruction. After each successful issue, the remaining code reads the operands and updates the RAT before considering the next instruction, so dependencies within one issue group are preserved. The shorter function signature uses the shared Machine object; this was a reorganization of the state as well as an added loop.

1.2 Select completed results for write-back

Initial Version

original_code/wb.py (excerpt)

```
def wb(cdb, rat_int, rat_fp,
       rs_int_adder, rs_fp_adder, rs_fp_multi,
       ld_sd_queue, ROB, cycle,
       results_buffer):
   # broadcast
   if cdb.valid == 1:
      boroadcast(cdb, rat_int, rat_fp,
                 rs_int_adder, rs_fp_adder, rs_fp_multi,
                 ld_sd_queue, ROB, cycle)
      cdb.valid = 0
   # fetch data from results queue
   if cdb.valid == 0:
      if len(results_buffer)>0:
         cdb.valid = 1
         cdb.value = results_buffer[0].value
         cdb.dest_tag = results_buffer[0].dest_tag
         results_buffer.popleft()
```

My Version

code/wb.py (excerpt)

```
def wb(m, cycle):
   finished = [e for e in m.rob if waiting_for_cdb(e, cycle)]
   for entry in sorted(finished, key=lambda e: e.seq)[:m.cfg.cdb_width]:
      broadcast(m, entry, cycle)
```

Explanation

The original write-back code uses one CDB object and fetches the next result from a buffer. My version selects completed instructions in age order and broadcasts up to cdb_width results. The default is still one CDB, independently of issue width, because the assignment specifies one CDB. waiting_for_cdb checks that EX, or MEM for a load, finished in an earlier cycle. broadcast records the result in the ROB and makes dependent operands usable in the following cycle. Raising the CDB count is an optional experiment, not a requirement for multiple issue.

1.3 Commit from the head of the ROB

Initial Version

original_code/commit.py (excerpt)

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):
    if len(ROB)>0:
        if instructions[ROB[0].PC].split(' ')[0]=='Bne':
            if len(ROB[0].exe)!=0:
                entry = ROB.popleft()
                print_ROB(entry, instructions)
        if (len(ROB)>0)&(instructions[ROB[0].PC].split(' ')[0]!='Bne'):
            if len(ROB[0].cdb)!=0: # broadcasted instructions
                ROB[0].commit.append(cycle+1)
                entry = ROB.popleft()
                modify_arch_reg(entry, reg_int, reg_fp)
                print_ROB(entry, instructions)
```

My Version

code/commit.py (excerpt)

```
def commit(m, cycle):
    for _ in range(m.cfg.commit_width):
        if not m.rob or not commit_head(m, m.rob[0], cycle):
            return
        entry = m.rob.popleft()
        entry.commit = cycle
        m.committed.append(entry)
```

Explanation

The original commit function handles the ROB head through separate branch, result and store cases. My version makes the retirement limit explicit with commit_width, which defaults to issue width. Each iteration checks the current head and stops immediately if it cannot commit, preserving program order. commit_head checks completion, updates the architectural register and clears a RAT mapping only when it still points to this instruction. A store also needs its address, data and the memory port; it writes memory at commit and checks whether younger loads need replay. These checks remain in commit_head rather than being duplicated in the loop.

1.4 Schedule execution using cycle timestamps

Initial Version

original_code/exe.py (excerpt)

```
def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu)!=0:
        for element in fu:
            element.cycle += 1
        # write starting cycle
        if fu[-1].cycle == 1:
            index = find_ROB_entry(ROB, fu[-1].dest_tag)
            ROB[index].exe.append(cycle)
        # finish cycle
        if fu[0].cycle == time_fu:
```

My Version

code/init.py (excerpt)

```
def can_start(self, cycle):
    if self.pipelined:
        return self.last_start != cycle
    return self.busy_until < cycle

def start(self, entry, cycle):
    self.last_start = cycle
    self.busy_until = cycle + self.latency - 1
    self.current = entry
    entry.ex_start = cycle
    entry.ex_end = cycle + self.latency - 1
```

code/exe.py (excerpt)

```
def exe(m, cycle):
    "'start new work'"
    for unit in ('int', 'fpadd', 'fpmul'):
        ready = [e for e in m.rs[unit]
                 if e is not None and e.issue < cycle
                 and e.op1.is_ready(cycle) and e.op2.is_ready(cycle)]
        for entry in dispatch(ready, m.fus[unit], cycle):
            m.rs[unit][m.rs[unit].index(entry)] = None
```

Explanation

The original functional-unit helper advances an internal cycle counter when it is called. My version records the start and end cycles when an instruction is dispatched. A pipelined unit can start one instruction per cycle; an unpipelined unit must finish its current instruction first. dispatch assigns the oldest ready instructions to available units. The readiness checks prevent a newly issued instruction from executing immediately and respect the cycle in which each operand becomes usable. This supports multiple functional units without advancing an instruction twice in one cycle.

1.5 Predict branches and recover after a wrong prediction

Initial Version

original_code/exe.py (excerpt)

```
elif (fu[0].op=='Bne'):
    fu_results.pop()
    if fu[0].value1 == fu[0].value2:
        PC.PC += 1
        PC.valid = 1
    else:
        index = find_ROB_entry(ROB, fu[0].dest_tag)
        offset = ROB[index].dest_tag
        PC.PC = int(PC.PC + 1 + offset/4)
        PC.valid = 1
```

My Version

code/issue.py (excerpt)

```
# function: BTB lookup, indexed by the low 3 bits of the word address
def predict(m, instr):
    entry = m.btb[instr.pc % BTB_SIZE]
    if entry is not None and entry['pc'] == instr.pc and entry['taken']:
        return entry['target']
```

code/exe.py (excerpt)

```
entry.actual_next = target if taken else instr.pc + 1
m.btb[instr.pc % BTB_SIZE] = {'pc': instr.pc, 'target': target, 'taken': taken}
if entry.actual_next != entry.pred_next:
    entry.mispredicted = True
    m.mispredictions += 1
    squash_after(m, entry.seq, entry.actual_next, cycle)
```

code/exe.py (excerpt)

```
# recover the RAT from the surviving ROB entries
m.rat = {}
for entry in m.rob:
    if entry.instr.dest is not None and entry.instr.dest != 'R0':
        m.rat[entry.instr.dest] = entry
# recovery takes this cycle + 1; correct fetch starts at cycle + 2
m.pc = new_pc
m.fetch_resume = cycle + 2
```

Explanation

The original issue code stops fetching at a Bne and waits for execution to update the PC. My version predicts at issue using an eight-entry, one-bit BTB, then checks the prediction at the end of EX. On a mismatch, squash_after removes younger instructions from the ROB, reservation stations, load/store queue and active units. It then rebuilds the RAT from surviving instructions and resumes fetch at cycle n+2. Internally the PC counts instructions, so instr.pc + 1 + instr.imm implements the assignment's word-offset branch rule. Both Beq and Bne are supported.

1.6 Check older stores before a load reads its value

Initial Version

original_code/mem.py (excerpt)

```
def forward_check_from_sd(ld_sd_queue, index):
    data = []
    for i in reversed(range(index)):
        if (ld_sd_queue[i].op=='Sd'):
            if (ld_sd_queue[i].address == ld_sd_queue[index].address):
                if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
                    data.append(ld_sd_queue[i].data)
            break
    return data
```

My Version

code/mem.py (excerpt)

```
older_stores = [s for s in m.ld_sd_queue[:i] if s.instr.is_store]
unresolved = any(s.addr is None or s.ex_end >= cycle for s in older_stores)
match = None
if not unresolved:
    for store in reversed(older_stores):
        if store.addr == entry.addr:
            match = store
            break
if match is not None:
    # forwarding from a store takes 1 cycle
    if match.op2.is_ready(cycle):
        entry.mem_start = entry.mem_end = cycle
        entry.result = match.op2.value
        entry.forwarded = True
        entry.source_seq = match.seq
    continue
if m.mem_busy_until < cycle:
    entry.mem_start = cycle
    entry.mem_end = cycle + m.cfg.mem_cycles - 1
    entry.result = m.memory.get(entry.addr, 0.0)
    m.mem_busy_until = entry.mem_end
```

Explanation

The original forwarding helper stops at the nearest older store, even if that store has a different address. My version first checks all older store addresses. If any address is unresolved, the load accesses memory when the port is free, following the assignment rule. Otherwise, it searches backward for the youngest matching store and forwards in one cycle when that store's data is ready. If no store matches, it reads memory. check_memory_order in exe.py checks for stale younger loads when a store address resolves and again when that store commits. A conflict replays the earliest affected load and all younger instructions; tests 09, 12 and 13 exercise these cases.

1.7 Organize the cycle loop and finish outstanding stores

Initial Version

original_code/main.py (excerpt)

```
while (len(ROB)>0)|(cycle==1):

    '''ISSUE stage'''
    if (PC.PC<len(instructions))&(PC.valid==1):
        issue(cycle, PC, instructions, ROB, size_ROB,
              rs_int_adder,
              rs_fp_adder,
              rs_fp_multi,
              ld_sd_queue, size_ld_sd_queue,
              rat_int, rat_fp)
```

My Version

code/main.py (excerpt)

```
cycle = 1
while True:
    mem(m, cycle)
    commit(m, cycle)
    wb(m, cycle)
    exe(m, cycle)
    issue(m, cycle)
    # done when everything has committed and the last store write has finished
    if not m.rob and m.pc >= len(m.program) and m.mem_busy_until <= cycle:
        return cycle
    cycle += 1
    if cycle > MAX_CYCLES:
        raise RuntimeError('no progress after %d cycles' % MAX_CYCLES)
```

Explanation

The original main loop starts with ISSUE and continues while the ROB is nonempty or it is the first cycle. My version calls MEM, COMMIT, WB, EX and ISSUE once per cycle. Explicit timestamps prevent an instruction from passing through dependent stages in the same cycle. MEM runs before COMMIT so a load can forward from a store that commits in that cycle; ISSUE runs last so it can reuse resources released during the cycle. The termination condition also waits for the memory port to finish the final store. Configuration and initial values now come from the input file through build_machine, and the completed run is passed to print_report for the timing, register and memory output.

Timing rules

- An instruction issued in cycle *c* can start EX in *c+1* at the earliest.
- A result broadcast on the CDB in cycle *w* can be used by a waiting instruction in *w+1*. An instruction commits in *w+1* at the earliest.
- A reservation station is freed when its instruction starts EX, and an instruction issued in that same cycle can take it. I used this convention because the midterm sample solution uses it.
- A load's MEM starts the cycle after its address is calculated. WB is the cycle after MEM ends.
- A store commits once its address and data are ready and the memory port is free. Its write then holds the port for Cycles in Mem cycles starting in the commit cycle, which is what the MEM column shows for stores.
- A branch commits the cycle after it is resolved. Branches and stores do not use the CDB.
- If an address or branch target is out of range, the error is only raised when that instruction reaches commit. A fault on a wrong path that gets squashed therefore has no effect.
- A memory-order violation can be caught when a store finishes calculating its address or when it commits. If detected in cycle n, the load and all younger instructions are squashed, cycle n+1 is used for recovery, and the load is fetched again in cycle n+2.
- The cycle count stops once every instruction has committed and the last store has finished writing to memory.
- Within a cycle the stages are called in the order MEM, COMMIT, WB, EX, ISSUE. Each value carries the cycle in which it becomes ready, so this order can't make a result available too early; it only decides who gets a shared resource first. Running MEM before COMMIT lets a load forward from a store that commits in the same cycle, and running ISSUE last lets it reuse a reservation station that EX freed in that cycle.

Branch offset

Branch offsets are in words, so target = PC + 4 + (offset << 2). The original test_case.txt had an offset of -12, which the old simulator treated as bytes. I changed it to -3 words so the branch still goes to the same place.

2. Validation

1. Final-state checks. run_tests.py runs all fourteen inputs at issue widths 1–4 (56 runs) and compares the registers, non-zero memory and the order of committed instructions with a sequential interpreter. All 56 runs pass. One caveat: the interpreter shares the parser and arithmetic helper with the simulator, so this check doesn't test those parts independently, and it doesn't check timing.

The extra memory-rule tests check that unresolved addresses take priority over a known forwarding match, that a bypassed store rechecks loads at commit, and that normal forwarding still takes one cycle. They run these three checks at every issue width. A fixed-seed generator also runs 30 mixed programs at every width and compares their final state and committed instruction order with the interpreter.

2. **Timing example.** Test 01 uses the setup from the midterm sample question on page 6: four-wide issue, four CDBs, four commits per cycle and four reservation stations of each type, with FP add taking 3 cycles, FP multiply 5 and memory access 6. The last instruction commits in cycle 33. The simulator's table below differs from the sample solution in three places.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2-2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	1	16-18	-	19	20
Mult.d F2 F1 F1	1	11-15	-	16	20
Add.d F4 F2 F4	2	17-19	-	20	21
Addi R1 R1 8	2	3-3	-	4	21
Ld F1 0(R1)	2	5-5	9-14	15	21
Mult.d F2 F1 F0	2	16-20	-	21	22
Add.d F3 F3 F2	3	22-24	-	25	26

Instruction	ISSUE	EX	MEM	WB	COMMIT
Mult.d F2 F1 F1	3	17-21	-	22	26
Add.d F4 F2 F4	3	23-25	-	26	27
Addi R1 R1 8	3	5-5	-	6	27
Ld F1 0(R1)	4	7-7	15-20	21	27
Mult.d F2 F1 F0	10	22-26	-	27	28
Add.d F3 F3 F2	16	28-30	-	31	32
Mult.d F2 F1 F1	16	23-27	-	28	32
Add.d F4 F2 F4	17	29-31	-	32	33

First, the sample solution has the second Addi executing in cycle 4. In my simulator a value broadcast in one cycle can be used in the next. The first Addi broadcasts R1 in cycle 4, so the second Addi can't execute before cycle 5 and broadcasts in cycle 6. Second, because of that, the third load computes its address in cycle 7, not 6. Third, the sample solution issues the last Add.d F4 in cycle 16. Only one add reservation station frees in cycle 16, and the Add.d F3 just before it takes that one; the next frees in cycle 17, so Add.d F4 issues in 17. None of this changes the final commit, which is still cycle 33.

3. Test cases

test	what it checks
test_case.txt	the given test case (loop over Addi/Bne, then one Add.d)
01_midterm_ooo	the midterm question above; RS-full stalls, a non-pipelined memory
02_straight_line	test case #01 of the sample report: straight-line code
03_loop_branch	a real 3-iteration loop (Bne back 8 instructions): BTB training, two mispredictions, squash and RAT recovery, load/store queue full
04_store_load_forwarding	store→load forwarding, a load to an unrelated address, a store whose data arrives late
05_structural_hazards	1 RS per unit, a 2-entry load/store queue, a 4-entry ROB
06_beq_loop	Beq forward exit plus an always-taken backward Beq; writes to R0 are ignored
07_multiple_fus	test case #02 of the sample report: 2 FP adders, 2 FP multipliers, 2 address adders
08_independent_ops	16 independent operations, 4 integer adders, 4 CDBs: high ILP
09_memory_violation	a load reads memory before an older store address is known; the store then matches, so the load is squashed and replayed
10_wrong_path_fault	a load with an out-of-range address on a mispredicted path is squashed without raising an error

test	what it checks
11_final_store	a store as the last instruction: the cycle count includes its memory write
12_unknown_with_match	A load reads memory despite a known matching store because another older address is unresolved; the conflict is replayed.
13_bypassed_known_store	An unrelated unresolved store makes a load bypass a known matching store; the matching store catches the stale load at commit.

4. Results under different issue widths

Total cycles at each issue width (every run also passes the correctness check). Totals include the last store's memory write, and each file sets its own number of CDBs (1 unless noted):

test	w=1	w=2	w=3	w=4
test_case.txt	41	35	34	34
01_midterm_ooo (4 CDBs)	35	33	33	33
02_straight_line	36	35	35	35
03_loop_branch	57	54	54	54
04_store_load_forwarding	24	23	23	23
05_structural_hazards	22	21	21	21
06_beq_loop	24	21	21	21
07_multiple_fus	47	46	46	46
08_independent_ops (4 CDBs)	20	12	10	8
09_memory_violation	19	19	19	19
10_wrong_path_fault	11	11	11	11
11_final_store	7	7	7	7
12_unknown_with_match	17	17	17	17
13_bypassed_known_store	30	29	29	29

Among the original twelve tests, going from width 1 to width 2 helps every test except 09–11, which are short programs held up by a dependency chain, a replayed load or one memory write. Test 12 also takes the same number of cycles at every width; test 13 improves by one cycle at width 2. Further increases usually give little benefit because instructions wait on dependencies, the memory port or the single CDB. For example, 02_straight_line takes 35 cycles at widths 2–4 with one CDB, and 33 cycles with four CDBs.

08_independent_ops is different: its instructions are independent and there are enough functional units, so wider issue pays off. It drops from 20 cycles at width 1 to 8 at width 4. Four instructions issue together each cycle, although the different latencies mean the later stages don't always handle four per cycle. Some of its rows:

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R5 R1 R2	1	2-2	-	3	4
Addi R6 R3 10	1	2-2	-	3	4
Sub R7 R4 R1	1	2-2	-	3	4
Add.d F3 F1 F2	1	2-3	-	4	5
Add R8 R2 R3	2	3-3	-	4	5
…					
Add R16 R3 R3	4	5-5	-	6	8

Branch example: 03_loop_branch, issue width 2

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F2 0(R1)	1	2-2	3-7	8	9
Mult.d F4 F2 F20	1	9-23	-	24	25
Ld F6 0(R2)	2	3-3	8-12	13	25
Add.d F6 F4 F6	2	25-28	-	29	30
Sd F6 0(R2)	3	4-4	33-37	-	33
Addi R1 R1 -4	3	4-4	-	5	33
Addi R2 R2 -4	4	5-5	-	6	34
Bne R1 R0 -8	4	6-6	-	-	34
Ld F2 0(R1)	8	9-9	13-17	18	35
…					
Bne R1 R0 -8	24	26-26	-	-	52
Add.d F20 F2 F2	28	29-32	-	35	52

- First Bne (cycle 4): the BTB is still empty, so it's predicted not taken and Add.d F20 is issued down the wrong path in cycle 5. The branch resolves as taken at the end of cycle 6, cycle 7 is spent recovering, and the correct path (Ld F2) issues in cycle 8, which is n+2.
- Second Bne: the BTB now says taken, so the prediction is right and no cycles are lost.
- Third Bne: predicted taken, but this is the loop exit, so it's actually not taken. It resolves in cycle 26 and the fall-through Add.d issues in cycle 28.
- The final state (Mem[40]=3, Mem[36]=16, Mem[32]=9, R1=0, R2=28) matches the reference interpreter.

The complete output for every test at every width (instruction table, registers, memory and statistics) is in results/.

5. What works / limitations

The simulator gives the correct final state in all 56 benchmark runs. The separate memory-rule tests also pass, including 12 directed checks and 120 generated-program runs. The tables above show the benchmark timings. The points below describe the load/store policy, fault handling and the cycle count.

Load/store behaviour and faults:

- Speculative loads: an unresolved older store makes the load access memory, even if another store has a known matching address. With all older addresses known, the load forwards from the youngest matching store or accesses memory when no match exists. Checking for stale loads at store commit is necessary because the matching store may have calculated its address before the load accessed memory. Tests 12 and 13 cover these cases; the load and its dependents are replayed before any stale result can commit.

- Cycle count: the simulation ends when the ROB is empty, fetching has finished and the memory port is idle. So a program whose last instruction is a store with a five-cycle write reports 7 cycles (commit in cycle 3, memory busy in cycles 3–7).
- Faults at commit: a bad load/store address or branch target is only recorded on the instruction. The error is raised if that instruction reaches the head of the ROB, so a faulting instruction on a mispredicted path is simply squashed (test 10).

Appendix B walks through each of these on a small input.

Other modelling choices:

- A branch misprediction is detected at the end of EX in cycle n, and nothing is issued in that cycle. Correct fetch starts at n+2, as the assignment requires. This only affects which wrong-path instructions show up in the squashed list.
- A load is replayed only after an actual address match. There's no prediction of which loads tend to conflict, so a load that keeps conflicting with an older store is replayed every time.
- The BTB is updated as soon as a branch resolves, even if that branch is later squashed by an older mispredicted branch. Real processors differ on this; I chose the simpler option.
- Integers are Python ints and FP values are Python floats, so single-precision rounding isn't modelled.
- Memory is a dictionary from byte address to value, with range checks. Byte-level storage and overlapping word accesses aren't modelled.

Appendix A: Tomasulo simulator source

These are the simulator and main test runner files from tomasulo-python/code/, as used for the results in this report. The additional regression checks are in code/test_memory_rules.py.

code/main.py

```
# Qing
# 21st May 2017
# Multi-issue version.
#
# usage: python main.py [input_file] [-w ISSUE_WIDTH] [-c CDB_WIDTH] [-s]
#   input_file   configuration + initial values + program (default: test_case.txt)
#   -w N         issue width 1-4, overrides "Issue width = N" in the file
#   -c N         number of CDBs, overrides "CDB width = N" in the file
#   -s           also list squashed instructions (wrong path or memory-order replay)

import argparse
from init import build_machine
from issue import issue
from exe import exe
from mem import mem
from wb import wb
from commit import commit
from print_status import print_report

MAX_CYCLES = 100000


def run(m):
    '''
    Every hand-off between stages is time-stamped (an operand broadcast in
    cycle c is usable in c+1, an instruction issued in c starts EX in c+1 at
    the earliest, ...), so the call order below only decides who wins a
    shared resource inside one cycle:
      - MEM before COMMIT: a load can still forward from a store that commits
        in the same cycle, and loads get the memory port before stores.
      - ISSUE last: an RS freed at EX start or a ROB entry freed at commit can
        be reused by an instruction issued in the same cycle.
    '''
    cycle = 1
    while True:
        mem(m, cycle)
        commit(m, cycle)
        wb(m, cycle)
        exe(m, cycle)
        issue(m, cycle)
        # done when everything has committed and the last store write has finished
        if not m.rob and m.pc >= len(m.program) and m.mem_busy_until <= cycle:
            return cycle
        cycle += 1
        if cycle > MAX_CYCLES:
            raise RuntimeError('no progress after %d cycles' % MAX_CYCLES)


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Multi-issue Tomasulo simulator')
    parser.add_argument('input', nargs='?', default='test_case.txt')
    parser.add_argument('-w', '--width', type=int, default=None)
    parser.add_argument('-c', '--cdb', type=int, default=None)
    parser.add_argument('-s', '--squashed', action='store_true')
    args = parser.parse_args()
    machine = build_machine(args.input, args.width, args.cdb)
    cycles = run(machine)
    print_report(machine, cycles, args.squashed)
```

code/init.py

```
# Qing
# 21st May 2017
# Multi-issue version: data structures, machine state and input-file parser.

import re
from collections import deque

'''functional unit class of every opcode'''
UNIT_OF = {
    'add': 'int', 'addi': 'int', 'sub': 'int', 'beq': 'int', 'bne': 'int',
    'add.d': 'fpadd', 'sub.d': 'fpadd',
    'mult.d': 'fpmul', 'mul.d': 'fpmul',
    'ld': 'ldsd', 'sd': 'ldsd',
}
UNIT_NAMES = {
    'integer adder': 'int',
    'fp adder': 'fpadd',
    'fp multiplier': 'fpmul',
    'load/store unit': 'ldsd',
}
BTB_SIZE = 8
MEMORY_SIZE = 256   # bytes


class Instruction:
    '''one decoded instruction; pc is its index in the program (byte PC = 4*pc)'''
    def __init__(self, pc, text):
        self.pc = pc
        self.text = ' '.join(text.split())
        tokens = [t for t in re.split(r'[,\s]+', text.strip()) if t]
        self.op = tokens[0].lower()
        if self.op not in UNIT_OF:
            raise ValueError('unknown instruction: ' + text)
        self.unit = UNIT_OF[self.op]
        self.dest = self.src1 = self.src2 = None
        self.imm = 0
        args = tokens[1:]
        if self.op in ('ld', 'sd'):
            m = re.match(r'^(-?\d+)\((\w+)\)$', args[1])
            if not m:
                raise ValueError('bad address operand: ' + text)
            self.imm = int(m.group(1))
            self.src1 = reg_name(m.group(2))          # base register
            if self.op == 'ld':
                self.dest = reg_name(args[0])
            else:
                self.src2 = reg_name(args[0])         # store data
        elif self.op in ('beq', 'bne'):
            self.src1, self.src2 = reg_name(args[0]), reg_name(args[1])
            self.imm = int(args[2])                   # offset in words
        elif self.op == 'addi':
            self.dest, self.src1 = reg_name(args[0]), reg_name(args[1])
            self.imm = int(args[2])
        else:
            self.dest = reg_name(args[0])
            self.src1, self.src2 = reg_name(args[1]), reg_name(args[2])

    @property
    def is_branch(self):
        return self.op in ('beq', 'bne')

    @property
    def is_load(self):
        return self.op == 'ld'

    @property
    def is_store(self):
        return self.op == 'sd'
```

```
def reg_name(token):
    token = token.upper()
    if not re.match(r'^[RF]([0-9]|[12][0-9]|3[01])$', token):
        raise ValueError('bad register: ' + token)
    return token


class Operand:
    '''a source operand: either a value (usable from cycle `ready`) or a ROB entry to wait for'''
    def __init__(self, value=0, src=None, ready=0):
        self.value = value
        self.src = src
        self.ready = ready

    def is_ready(self, cycle):
        return self.src is None and self.ready <= cycle


class ROB_entry:
    '''one in-flight instruction; it is also the RS / ld-sd queue entry'''
    def __init__(self, seq, tag, instr):
        self.seq = seq                # global age, smaller = older
        self.tag = tag
        self.instr = instr
        self.op1 = Operand()          # src1 (base register for ld/sd)
        self.op2 = Operand()          # src2 (store data for sd)
        self.result = None            # computed at end of EX / MEM
        self.value = None             # broadcast on the CDB
        self.addr = None              # ld/sd effective address
        self.forwarded = False        # ld got its value from a store
        self.source_seq = -1          # ld: seq of the store it forwarded from, -1 = memory
        self.fault = None             # bad address / branch target, raised at commit
        self.squashed = False
        # branch
        self.pred_next = None
        self.actual_next = None
        self.mispredicted = False
        # timing
        self.issue = None
        self.ex_start = self.ex_end = None
        self.mem_start = self.mem_end = None
        self.wb = None
        self.commit = None


class FunctionalUnit:
    def __init__(self, latency, pipelined):
        self.latency = latency
        self.pipelined = pipelined
        self.busy_until = 0           # unpipelined: last EX cycle of current op
        self.last_start = 0           # pipelined: accepts one new op per cycle
        self.current = None

    def can_start(self, cycle):
        if self.pipelined:
            return self.last_start != cycle
        return self.busy_until < cycle

    def start(self, entry, cycle):
        self.last_start = cycle
        self.busy_until = cycle + self.latency - 1
        self.current = entry
        entry.ex_start = cycle
        entry.ex_end = cycle + self.latency - 1


class Config:
    def __init__(self):
        # unit: [# of rs, cycles in EX, # of FUs]; ldsd also has cycles in MEM
        self.rs = {'int': 2, 'fpadd': 3, 'fpmul': 2, 'ldsd': 3}
        self.ex = {'int': 1, 'fpadd': 3, 'fpmul': 20, 'ldsd': 1}
        self.fus = {'int': 1, 'fpadd': 1, 'fpmul': 1, 'ldsd': 1}
        self.mem_cycles = 4
        self.rob_size = 128
```

```
        self.issue_width = 1
        self.cdb_width = 1              # the spec has one CDB
        self.commit_width = None        # defaults to the issue width


class Machine:
    '''all processor state'''
    def __init__(self, cfg, program, reg_init, mem_init):
        self.cfg = cfg
        self.program = program
        self.reg_int = [0] * 32
        self.reg_fp = [0.0] * 32
        for reg, value in reg_init.items():
            if reg == 'R0':
                continue
            if reg[0] == 'R':
                self.reg_int[int(reg[1:])] = int(value)
            else:
                self.reg_fp[int(reg[1:])] = float(value)
        self.memory = dict(mem_init)
        self.rat = {}                   # register -> ROB_entry; absent = register file
        self.rob = deque()
        self.rs = {u: [None] * cfg.rs[u] for u in ('int', 'fpadd', 'fpmul')}
        self.ld_sd_queue = []           # ROB entries of loads/stores, program order
        self.fus = {
            'int': [FunctionalUnit(cfg.ex['int'], False) for _ in range(cfg.fus['int'])],
            'fpadd': [FunctionalUnit(cfg.ex['fpadd'], True) for _ in range(cfg.fus['fpadd'])],
            'fpmul': [FunctionalUnit(cfg.ex['fpmul'], True) for _ in range(cfg.fus['fpmul'])],
            # dedicated effective-address adders
            'ldsd': [FunctionalUnit(cfg.ex['ldsd'], False) for _ in range(cfg.fus['ldsd'])],
        }
        self.mem_busy_until = 0         # memory: single-ported, non-pipelined
        self.mem_owner = None
        self.btb = [None] * BTB_SIZE   # {'pc', 'target', 'taken'} (1-bit predictor)
        self.pc = 0
        self.fetch_resume = 1           # first cycle fetch may continue after a squash
        self.next_seq = 0
        self.committed = []
        self.squashed = []
        self.branches = 0
        self.mispredictions = 0
        self.memory_violations = 0

    def reg_value(self, reg):
        if reg[0] == 'R':
            return self.reg_int[int(reg[1:])]
        return self.reg_fp[int(reg[1:])]

    def set_reg(self, reg, value):
        if reg == 'R0':
            return
        if reg[0] == 'R':
            self.reg_int[int(reg[1:])] = int(value)
        else:
            self.reg_fp[int(reg[1:])] = float(value)


# function: read input file (configuration, initial values, program)
def read_input(path):
    cfg = Config()
    program, reg_init, mem_init = [], {}, {}
    with open(path) as f:
        lines = f.read().splitlines()
    for raw in lines:
        line = raw.split('#')[0].strip()
        if not line:
            continue
        low = line.lower()
        name = next((n for n in UNIT_NAMES if low.startswith(n)), None)
        if name:
            unit = UNIT_NAMES[name]
            nums = [int(x) for x in low[len(name):].split()]
            if unit == 'ldsd':
                cfg.rs[unit], cfg.ex[unit], cfg.mem_cycles, cfg.fus[unit] = nums
```

```
            else:
                cfg.rs[unit], cfg.ex[unit], cfg.fus[unit] = nums
            continue
        m = re.match(r'^rob entries\s*=\s*(\d+)$', low)
        if m:
            cfg.rob_size = int(m.group(1))
            continue
        m = re.match(r'^issue width\s*=\s*(\d+)$', low)
        if m:
            cfg.issue_width = int(m.group(1))
            continue
        m = re.match(r'^cdb (?:width|buses)\s*=\s*(\d+)$', low)
        if m:
            cfg.cdb_width = int(m.group(1))
            continue
        m = re.match(r'^commit width\s*=\s*(\d+)$', low)
        if m:
            cfg.commit_width = int(m.group(1))
            continue
        if '=' in line:
            # R1=10, F2=30.1  or  Mem[4]=1, Mem[8]=2
            for item in line.split(','):
                name, value = [s.strip() for s in item.split('=')]
                mm = re.match(r'^mem\[(\d+)\]$', name.lower())
                if mm:
                    mem_init[int(mm.group(1))] = float(value)
                else:
                    reg_init[reg_name(name)] = float(value)
            continue
        program.append(Instruction(len(program), line))
    return cfg, program, reg_init, mem_init


# function: build machine; issue_width / cdb_width arguments override the file
def build_machine(path, issue_width=None, cdb_width=None):
    cfg, program, reg_init, mem_init = read_input(path)
    if issue_width is not None:
        cfg.issue_width = issue_width
    if cdb_width is not None:
        cfg.cdb_width = cdb_width
    if not 1 <= cfg.issue_width <= 4:
        raise ValueError('issue width must be 1-4')
    if cfg.commit_width is None:
        cfg.commit_width = cfg.issue_width
    return Machine(cfg, program, reg_init, mem_init)
```

code/issue.py

```
# Qing
# 21st May 2017
# Multi-issue version.

"""
Issue up to `issue_width` instructions per cycle, in program order:
1. stop at the first instruction that has no free ROB entry / RS / ld-sd queue entry
2. read operands through the RAT, put the instruction in ROB and RS / ld-sd queue
3. rename the destination in the RAT
4. branches: predict with the BTB (1-bit predictor); a predicted-taken branch
   ends the issue group and fetch continues at the target in the next cycle
"""
from init import ROB_entry, Operand, BTB_SIZE


# function: read a source register through the RAT
def read_operand(m, reg):
    if reg is None or reg == 'R0':
        return Operand(0)
    producer = m.rat.get(reg)
    if producer is None:
        return Operand(m.reg_value(reg))
    if producer.wb is not None:
        # already broadcast: value sits in the ROB
        return Operand(producer.value, None, producer.wb + 1)
    return Operand(None, producer)


# function: find a free reservation station
def check_rs_space(station):
    for i, entry in enumerate(station):
        if entry is None:
            return i
    return -1


# function: BTB lookup, indexed by the low 3 bits of the word address
def predict(m, instr):
    entry = m.btb[instr.pc % BTB_SIZE]
    if entry is not None and entry['pc'] == instr.pc and entry['taken']:
        return entry['target']
    return instr.pc + 1


def issue(m, cycle):
    if cycle < m.fetch_resume:
        return
    for _ in range(m.cfg.issue_width):
        if m.pc >= len(m.program):
            return
        instr = m.program[m.pc]
        # structural hazards stall issue (in order)
        if len(m.rob) >= m.cfg.rob_size:
            return
        if instr.unit == 'ldsd':
            if len(m.ld_sd_queue) >= m.cfg.rs['ldsd']:
                return
        else:
            index = check_rs_space(m.rs[instr.unit])
            if index < 0:
                return

        entry = ROB_entry(m.next_seq, 'ROB%d' % (m.next_seq % m.cfg.rob_size), instr)
        m.next_seq += 1
        entry.issue = cycle
        entry.op1 = read_operand(m, instr.src1)
        entry.op2 = read_operand(m, instr.src2)
        m.rob.append(entry)
        if instr.unit == 'ldsd':
            m.ld_sd_queue.append(entry)
```

```
        else:
            m.rs[instr.unit][index] = entry
        if instr.dest is not None and instr.dest != 'R0':
            m.rat[instr.dest] = entry

        if instr.is_branch:
            m.branches += 1
            entry.pred_next = predict(m, instr)
            m.pc = entry.pred_next
            if entry.pred_next != instr.pc + 1:
                return      # predicted taken: fetch target next cycle
        else:
            m.pc += 1
```

code/exe.py

```
# Qing
# 24th May 2017
# Multi-issue version.

'''
1. start ready instructions on free functional units, oldest first
   -- an RS is released when its instruction starts EX
   -- ld/sd compute their address on the dedicated address adders
2. finish instructions whose EX ends this cycle
   -- ALU: compute the result (broadcast later in WB)
   -- ld/sd: write the address into the ld/sd queue; a store whose address is
      now known checks younger loads that already read a stale value
   -- branch: resolve, train the BTB, squash on a misprediction
3. a bad address or branch target only marks the instruction; the error is
   raised if it reaches commit, so a wrong-path fault is simply squashed
'''
from collections import deque
from init import BTB_SIZE, MEMORY_SIZE


# function: compute an ALU result
def alu(op, a, b):
    if op in ('add', 'addi'):
        return int(a + b)
    if op == 'sub':
        return int(a - b)
    if op == 'add.d':
        return a + b
    if op == 'sub.d':
        return a - b
    if op in ('mult.d', 'mul.d'):
        return a * b
    raise ValueError(op)


# function: give each ready instruction a free unit, oldest first
def dispatch(entries, units, cycle):
    started = []
    for entry in sorted(entries, key=lambda e: e.seq):
        unit = next((u for u in units if u.can_start(cycle)), None)
        if unit is None:
            break
        unit.start(entry, cycle)
        started.append(entry)
    return started


# function: squash every instruction younger than `keep_seq`, refetch at `new_pc`
def squash_after(m, keep_seq, new_pc, cycle):
    for entry in m.rob:
        if entry.seq > keep_seq:
            entry.squashed = True
            m.squashed.append(entry)
    m.rob = deque(e for e in m.rob if not e.squashed)
    # clear reservation stations, ld/sd queue, busy units
    for unit in m.rs:
        m.rs[unit] = [None if (e is not None and e.squashed) else e for e in m.rs[unit]]
    m.ld_sd_queue = [e for e in m.ld_sd_queue if not e.squashed]
    for units in m.fus.values():
        for u in units:
            if u.current is not None and u.current.squashed:
                u.busy_until = cycle
                u.current = None
    if m.mem_owner is not None and m.mem_owner.squashed:
        m.mem_busy_until = cycle
        m.mem_owner = None
    # recover the RAT from the surviving ROB entries
    m.rat = {}
    for entry in m.rob:
        if entry.instr.dest is not None and entry.instr.dest != 'R0':
```

```
            m.rat[entry.instr.dest] = entry
    # recovery takes this cycle + 1; correct fetch starts at cycle + 2
    m.pc = new_pc
    m.fetch_resume = cycle + 2


# function: at store address resolution or commit, find stale younger loads
def check_memory_order(m, store, cycle):
    stale = [e for e in m.rob
             if e.instr.is_load and e.seq > store.seq and e.mem_start is not None
             and e.addr == store.addr and e.source_seq < store.seq]
    if stale:
        load = min(stale, key=lambda e: e.seq)
        m.memory_violations += 1
        squash_after(m, load.seq - 1, load.instr.pc, cycle)


# function: resolve a branch at the end of EX
def resolve_branch(m, entry, cycle):
    instr = entry.instr
    a, b = entry.op1.value, entry.op2.value
    taken = (a == b) if instr.op == 'beq' else (a != b)
    target = instr.pc + 1 + instr.imm
    if taken and target < 0:
        # fault: stop fetching; raised at commit unless an older branch squashes it
        entry.fault = 'branch target out of range in "%s"' % instr.text
        squash_after(m, entry.seq, len(m.program), cycle)
        return
    entry.actual_next = target if taken else instr.pc + 1
    m.btb[instr.pc % BTB_SIZE] = {'pc': instr.pc, 'target': target, 'taken': taken}
    if entry.actual_next != entry.pred_next:
        entry.mispredicted = True
        m.mispredictions += 1
        squash_after(m, entry.seq, entry.actual_next, cycle)


def exe(m, cycle):
    '''start new work'''
    for unit in ('int', 'fpadd', 'fpmul'):
        ready = [e for e in m.rs[unit]
                 if e is not None and e.issue < cycle
                 and e.op1.is_ready(cycle) and e.op2.is_ready(cycle)]
        for entry in dispatch(ready, m.fus[unit], cycle):
            m.rs[unit][m.rs[unit].index(entry)] = None
    ready = [e for e in m.ld_sd_queue
             if e.ex_start is None and e.issue < cycle and e.op1.is_ready(cycle)]
    dispatch(ready, m.fus['ldsd'], cycle)

    '''finish work whose EX ends this cycle'''
    for entry in sorted(m.rob, key=lambda e: e.seq):
        if entry.squashed or entry.ex_end != cycle:
            continue
        instr = entry.instr
        if instr.unit == 'ldsd':
            addr = entry.op1.value + instr.imm
            if 0 <= addr < MEMORY_SIZE:
                entry.addr = addr
                if instr.is_store:
                    check_memory_order(m, entry, cycle)
            else:
                entry.fault = 'address %d out of range in "%s"' % (addr, instr.text)
        elif instr.is_branch:
            resolve_branch(m, entry, cycle)
        else:
            b = instr.imm if instr.op == 'addi' else entry.op2.value
            entry.result = alu(instr.op, entry.op1.value, b)
```

code/mem.py

```
# Qing
# 26th May 2017
# Multi-issue version.

'''
MEM stage for loads (stores write memory at commit, see commit.py)
1. a load whose address is ready checks the older stores in the ld/sd queue
   -- any address unresolved: access memory, even if another store matches
   -- all addresses known: forward from the youngest matching store (1 cycle),
      waiting for its data if needed; if no match exists, access memory
   -- memory access uses the single, non-pipelined port for `mem_cycles` cycles
2. stores check for stale younger loads both when their address resolves and
   when they commit; stale loads and their younger instructions are replayed
3. a load leaves the ld/sd queue once it has its data
'''


def mem(m, cycle):
    # loads that got their data in an earlier cycle leave the queue
    m.ld_sd_queue = [e for e in m.ld_sd_queue
                     if not (e.instr.is_load and e.mem_end is not None and e.mem_end < cycle)]

    for i, entry in enumerate(m.ld_sd_queue):
        if not entry.instr.is_load or entry.mem_start is not None:
            continue
        if entry.addr is None or entry.ex_end >= cycle:
            continue
        older_stores = [s for s in m.ld_sd_queue[:i] if s.instr.is_store]
        unresolved = any(s.addr is None or s.ex_end >= cycle for s in older_stores)
        match = None
        if not unresolved:
            for store in reversed(older_stores):
                if store.addr == entry.addr:
                    match = store
                    break
        if match is not None:
            # forwarding from a store takes 1 cycle
            if match.op2.is_ready(cycle):
                entry.mem_start = entry.mem_end = cycle
                entry.result = match.op2.value
                entry.forwarded = True
                entry.source_seq = match.seq
            continue
        if m.mem_busy_until < cycle:
            entry.mem_start = cycle
            entry.mem_end = cycle + m.cfg.mem_cycles - 1
            entry.result = m.memory.get(entry.addr, 0.0)
            m.mem_busy_until = entry.mem_end
            m.mem_owner = entry
```

code/wb.py

```
# Qing
# 29th May 2017
# Multi-issue version.

'''
1. pick finished instructions, oldest first, up to `cdb_width` per cycle
   -- ALU ops: the cycle after EX ends; loads: the cycle after MEM ends
2. broadcast: write the value into the ROB and wake up every waiting operand
   (reservation stations, ld/sd queue base registers and store data)
'''


# function: has the instruction finished and is waiting for the CDB
def waiting_for_cdb(entry, cycle):
    instr = entry.instr
    if entry.wb is not None or instr.is_branch or instr.is_store:
        return False
    if instr.is_load:
        return entry.mem_end is not None and entry.mem_end < cycle
    return entry.ex_end is not None and entry.ex_end < cycle


# function: broadcast one result
def broadcast(m, producer, cycle):
    producer.wb = cycle
    producer.value = producer.result
    for entry in m.rob:
        for op in (entry.op1, entry.op2):
            if op.src is producer:
                op.value = producer.value
                op.src = None
                op.ready = cycle + 1


def wb(m, cycle):
    finished = [e for e in m.rob if waiting_for_cdb(e, cycle)]
    for entry in sorted(finished, key=lambda e: e.seq)[:m.cfg.cdb_width]:
        broadcast(m, entry, cycle)
```

code/commit.py

```
# Qing
# 29th may 2017
# Multi-issue version.
'''
Commit up to `commit_width` instructions per cycle from the ROB head, in order
    -- ALU / load: the cycle after WB; write the register file, free the RAT entry
    -- branch: the cycle after it is resolved
    -- store: address and data ready and the memory port free; the store
       writes memory starting in its commit cycle (MEM column) and leaves
       the ld/sd queue
    -- an instruction with a recorded fault raises the error here
'''
from exe import check_memory_order


# function: try to commit the ROB head; return True on success
def commit_head(m, entry, cycle):
    instr = entry.instr
    if entry.fault is not None:
        # precise exception: only an instruction that reaches commit faults
        raise RuntimeError('cycle %d: %s' % (cycle, entry.fault))
    if instr.is_branch:
        return entry.ex_end is not None and entry.ex_end < cycle
    if instr.is_store:
        if entry.addr is None or entry.ex_end >= cycle:
            return False
        if not entry.op2.is_ready(cycle) or m.mem_busy_until >= cycle:
            return False
        entry.mem_start = cycle
        entry.mem_end = cycle + m.cfg.mem_cycles - 1
        m.mem_busy_until = entry.mem_end
        m.mem_owner = entry
        m.memory[entry.addr] = entry.op2.value
        m.ld_sd_queue.remove(entry)
        # A load may have bypassed this known address because a different
        # older store was unresolved. Catch it before any stale load retires.
        check_memory_order(m, entry, cycle)
        return True
    if entry.wb is None or entry.wb >= cycle:
        return False
    if instr.dest is not None:
        m.set_reg(instr.dest, entry.value)
        if m.rat.get(instr.dest) is entry:
            del m.rat[instr.dest]
    return True


def commit(m, cycle):
    for _ in range(m.cfg.commit_width):
        if not m.rob or not commit_head(m, m.rob[0], cycle):
            return
        entry = m.rob.popleft()
        entry.commit = cycle
        m.committed.append(entry)
```

code/print_status.py

```
# Qing
# 21st May 2017
# Multi-issue version: final report (instruction status, registers, memory).


def span(start, end):
    if start is None:
        return '-'
    return '%d-%d' % (start, end)


def cell(value):
    return '-' if value is None else str(value)


def fmt(value):
    if isinstance(value, float):
        return '%g' % value
    return str(value)


def print_report(m, cycles, show_squashed=False):
    cfg = m.cfg
    print('Issue width = %d, CDB width = %d, commit width = %d, ROB entries = %d'
          % (cfg.issue_width, cfg.cdb_width, cfg.commit_width, cfg.rob_size))
    print()
    print(''.ljust(26) + 'ISSUE'.ljust(8) + 'EX'.ljust(10) + 'MEM'.ljust(10)
          + 'WB'.ljust(8) + 'COMMIT')
    for e in m.committed:
        print(e.instr.text.ljust(26) + cell(e.issue).ljust(8)
              + span(e.ex_start, e.ex_end).ljust(10)
              + span(e.mem_start, e.mem_end).ljust(10)
              + cell(e.wb).ljust(8) + cell(e.commit))
    if show_squashed and m.squashed:
        print()
        print('Squashed instructions (wrong path or memory-order replay):')
        for e in m.squashed:
            print('  ' + e.instr.text.ljust(24) + 'issued in cycle %d' % e.issue)

    print()
    print('REGISTER VALUES')
    for kind, regs in (('R', m.reg_int), ('F', m.reg_fp)):
        for base in range(0, 32, 8):
            print(''.ljust(7) + ''.join(('%s%d' % (kind, i)).ljust(9) for i in range(base, base + 8)))
            print('value: ' + ''.join(fmt(regs[i]).ljust(9) for i in range(base, base + 8)))

    print()
    print('NON-ZERO MEMORY VALUES')
    print('Addresses'.ljust(12) + 'Values')
    for addr in sorted(m.memory):
        if m.memory[addr] != 0:
            print(str(addr).ljust(12) + fmt(m.memory[addr]))

    print()
    n = len(m.committed)
    print('Cycles: %d   Committed instructions: %d   IPC: %.3f'
          % (cycles, n, n / cycles if cycles else 0))
    print('Branches issued: %d   Mispredicted: %d   Memory-order violations: %d   '
          'Squashed instructions: %d'
          % (m.branches, m.mispredictions, m.memory_violations, len(m.squashed)))
```

code/run_tests.py

```
# Correctness check for every test case at issue widths 1-4.
#
# Each input is also executed by a plain in-order interpreter (no timing).
# The out-of-order machine must end with the same registers and memory and
# commit exactly the same instruction sequence, at every issue width.
#
# usage: python run_tests.py [input_file ...]   (default: test_case.txt tests/*.txt)

import glob
import sys
from init import build_machine
from exe import alu
from main import run


def reference(path):
    m = build_machine(path, 1)
    trace = []
    pc = 0
    while pc < len(m.program):
        instr = m.program[pc]
        trace.append(pc)
        if len(trace) > 100000:
            raise RuntimeError('reference run does not terminate')
        a = m.reg_value(instr.src1) if instr.src1 else 0
        b = m.reg_value(instr.src2) if instr.src2 else 0
        if instr.is_branch:
            taken = (a == b) if instr.op == 'beq' else (a != b)
            pc = pc + 1 + instr.imm if taken else pc + 1
            continue
        if instr.is_load:
            m.set_reg(instr.dest, m.memory.get(a + instr.imm, 0.0))
        elif instr.is_store:
            m.memory[a + instr.imm] = b
        else:
            m.set_reg(instr.dest, alu(instr.op, a, instr.imm if instr.op == 'addi' else b))
        pc += 1
    return m, trace


def nonzero(memory):
    return {a: v for a, v in memory.items() if v != 0}


def main(paths):
    failures = 0
    print('%-36s %6s %6s %6s %6s   %s' % ('test', 'w=1', 'w=2', 'w=3', 'w=4', '(cycles)'))
    for path in paths:
        ref, trace = reference(path)
        cycles = []
        for width in (1, 2, 3, 4):
            m = build_machine(path, width)
            n = run(m)
            ok = (m.reg_int == ref.reg_int and m.reg_fp == ref.reg_fp
                  and nonzero(m.memory) == nonzero(ref.memory)
                  and [e.instr.pc for e in m.committed] == trace)
            cycles.append(str(n) if ok else 'FAIL')
            failures += not ok
        print('%-36s %6s %6s %6s %6s' % ((path,) + tuple(cycles)))
    print('all tests passed' if failures == 0 else '%d failures' % failures)
    return failures


if __name__ == '__main__':
    sys.exit(main(sys.argv[1:] or ['test_case.txt'] + sorted(glob.glob('tests/*.txt'))))
```

Appendix B: Additional verification cases

I ran each of these small cases at issue widths 1 and 4 with the final code, and both widths give the results below. Similar programs are included in the test suite as tests 09–11.

B1. Final-store completion

```
Load/store unit 3 1 5 1
F1=7
Sd F1 0(R0)
```

Result: ISSUE 1, EX 2, MEM 3–7, COMMIT 3, and 7 cycles in total. Address 0 ends up holding 7. The total includes the store's memory write.

B2. A faulting load on the wrong path

```
Integer adder 4 3 1
Load/store unit 3 1 5 1
Beq R0 R0 1
Ld F1 256(R0)
Addi R2 R0 7
```

The branch is always taken, so the load should never run. But the BTB is empty at first, so the branch is predicted not taken and the load (address 256, out of range) is issued and executed on the wrong path. Its fault is only recorded. When the branch resolves in cycle 4 the load is squashed, and the program finishes in 11 cycles with R2 = 7.

B3. An older store with an unresolved address

```
Integer adder 4 4 1
Load/store unit 4 1 3 1
F1=9
Mem[4]=2
Addi R1 R0 4
Sd F1 0(R1)
Ld F2 4(R0)
```

The load doesn't wait for the store's address. At width 1 it reads memory in cycles 5–7 and gets the old value, 2. In cycle 7 the store's address is calculated as 4, which matches, so the load is squashed and fetched again in cycle 9. Meanwhile the store commits in cycle 8 and writes memory in cycles 8–10, so the replayed load reads 9 in cycles 11–13. The program finishes in 15 cycles with F2 = 9 and one memory-order violation. At width 4 the first load issues in cycle 1, but the replay and everything after it happen in the same cycles.